If you set the `:optional` option to `true`, then the presence of the associated object won't be validated. By default, this option is set to `false`.

4.1.3 Scopes for `belongs_to`

There may be times when you wish to customize the query used by `belongs_to`. Such customizations can be achieved via a scope block. For example:

```
class Order < ActiveRecord::Base
  belongs_to :customer, -> { where active: true },
                        dependent: :destroy
end
```

You can use any of the standard querying methods inside the scope block. The following ones are discussed below:

- `where`
- `includes`
- `readonly`
- `select`

4.1.3.1 `where`

The `where` method lets you specify the conditions that the associated object must meet.

```
class Order < ActiveRecord::Base
  belongs_to :customer, -> { where active: true }
end
```

4.1.3.2 `includes`

You can use the `includes` method to specify second-order associations that should be eager-loaded when this association is used. For example, consider these models:

```
class LineItem < ActiveRecord::Base
  belongs_to :order
end

class Order < ActiveRecord::Base
  belongs_to :customer
  has_many :line_items
end

class Customer < ActiveRecord::Base
  has_many :orders
end
```

If you frequently retrieve customers directly from line items (`@line_item.order.customer`), then you can make your code somewhat more efficient by including customers in the association from line items to orders:

```
class LineItem < ActiveRecord::Base
  belongs_to :order, -> { includes :customer }
end

class Order < ActiveRecord::Base
  belongs_to :customer
  has_many :line_items
end

class Customer < ActiveRecord::Base
  has_many :orders
end
```

There's no need to use `includes` for immediate associations - that is, if you have `Order belongs_to :customer`, then the customer is eager-loaded automatically when it's needed.

4.1.3.3 `readonly`

If you use `readonly`, then the associated object will be read-only when retrieved via the association.

4.1.3.4 `select`